



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

多线程编程实验

林晖鹏

年级：2023 级

专业：计算机科学与技术

指导教师：王刚

2025 年 6 月 29 日

摘要

本文基于学校提供的服务器及 MPI 节点，实现了多线程并行程序设计实验，重点研究了基于消息传递接口（MPI）的并行优化方案及流水线并行策略。实验通过任务划分、OpenMP 并行化以及 SIMD 优化，完成了口令猜测任务的并行实现，并分析了通信与计算失衡对性能的影响。基础 MPI 实现中，因频繁阻塞通信导致性能下降，O2 优化下通信时间占比高达 43.99%，加速比仅 0.08×；通过禁用 memcpy 优化，通信时间显著降低。流水线优化采用两进程分别处理猜测生成与哈希计算，结合 OpenMP 并行化字符串处理，在 O2 优化（禁用 memcpy）下实现 Guess+Hash 时间从 3.76 秒降至 3.04 秒，加速比达 1.24×。实验验证了 MPI 流水线结合 OpenMP 的多层次并行优化有效性，并提出了非阻塞通信等改进方向。代码仓库地址：<https://gitee.com/absurdaaa/parallel-formal-warehouse>。（相关代码在服务器上面 guess/MPI_base 和 guess/MPI_pipeline 里面。）

关键字：并行程序设计，MPI，OpenMP，流水线优化，通信开销，口令猜测，性能优化

目录

一、 基础要求	1
(一) MPI 简介	1
(二) 基础代码 MPI 实现	1
1. 实现思路	1
2. 代码实现	1
3. 运行结果	6
4. 阻塞时间差异探究	7
二、 进阶要求	8
(一) 多 PT 并行实现	8
(二) guess、hash 流水线实现	8
1. 实现思路	8
2. 代码实现	8
3. 结果分析	11
三、 总结	11

一、 基础要求

(一) MPI 简介

消息传递接口 (MPI) 是一种用于并行计算的标准化通信协议, 用于在分布式内存系统中实现进程间高效数据交换。MPI 提供了一个可移植且可扩展的框架, 广泛应用于科学计算领域的高性能并行程序开发。它支持点对点和集体通信操作, 使进程能够有效协调任务。

(二) 基础代码 MPI 实现

1. 实现思路

下面我们对原本代码中 TODO 的部分进行 MPI 并行实现, 思路相当简单:

1. 前面部分每个进程都同时做, 保证每个进程的信息相同, 这样能避免过多的信息传递
2. 在要求 TODO 的部分开始并行处理, 每个进程负责一部分的 guess 生成, 然后最后汇总到 rank0 进程里汇总。
3. 最后 hash 部分也只需要 rank0 进程处理, 因为其他进程的对象的 guesses 不会增加。
4. 注意这里时间测量要换成 MPI 库内置的计时函数。

2. 代码实现

1. 在 main 函数开头 MPI 初始化, 并获取进程编号。

```
1 // 初始化MPI
2 MPI_Init(&argc, &argv);
3 int rank, size;
4 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5 MPI_Comm_size(MPI_COMM_WORLD, &size);
```

2. 在 TODO 位置我们换成 MPI 实现, 这里添加了一个函数代替。参数分别是要拼接的 string 字符串、要处理的个数、目标 guesses 向量、guess 前缀。

```
1 // for (int i = 0; i < pt.max_indices[pt.content.size() - 1]; i += 1)
2 // {
3 //     string temp = guess + a->ordered_values[i];
4 //     guesses.emplace_back(temp);
5 //     total_guesses += 1;
6 // }
7
8 MPI(a->ordered_values, pt.max_indices[pt.content.size() - 1], guesses,
    guess);
```

3. 下面主要介绍 MPI 函数的流程:

(1) 获取当前进程的 rank 编号, 并且划分每个进程要处理的字符串的下标范围。如果无法均分, 那么前几个进程将会多一个要处理的串, 这样能保证尽可能的每个进程的花销相近, 这样能减少阻塞等待的时间。

```

1  int rank, size;
2  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
3  MPI_Comm_size(MPI_COMM_WORLD, &size);
4  // 划分任务
5  int chunk = num / size; // 每个进程分配的任务大小
6  int remainder = num % size; // 剩余任务大小
7  int start = rank * chunk + min(rank, remainder);
8  int end = start + chunk + (rank < remainder ? 1 : 0);

```

(2) 为了后面一个进程里汇总的时候能够并行写入，我们这里在除了记录每个处理后的 string 有多长，还顺便算了前缀和来记录每个在汇总字符串里面的偏移量。计算了总长度，这样总字符串初始化的时候就能直接预设大小，避免不够空间时变长导致的内存移动时间开销。

```

1  int sub_num = end - start;
2  int local_total_len = 0;
3  std::vector<int> local_lengths(sub_num);
4
5  // 记录偏移量，这样可以并行拼接
6  std::vector<int> local_offsets(sub_num);
7
8  // 先检查自己的任务所有 string 的长度加起来有多少，同时记录每个的偏移量
9  if(guess!=""){
10     int len = guess.length();
11     for (int i = start; i < end; i++)
12     {
13         local_offsets[i-start] = local_total_len;
14         local_total_len += val[i].length() + len;
15         local_lengths[i-start] = len + val[i].length();
16     }
17 }
18 else{
19     for (int i = start; i < end; i++)
20     {
21         local_offsets[i - start] = local_total_len;
22         local_total_len += val[i].length();
23         local_lengths[i - start] = val[i].length();
24     }
25 }

```

(3) 每个线程各自处理一部分字符串，并本地存储在一个 char 的向量里面（因为 MPI 只能传递固定内存长度的连续内存，所以不能直接传 vector<string> 这样变长的，所以这里我们用 char 来储存）。由于前面我们计算了每个字符串写入汇总字符串的偏移量，所以这里我们可以用 openmp 加速并行汇总！

```

1  std::vector<char> local_chars(local_total_len);
2
3  if(guess!=""){

```

```

4 #pragma omp parallel for
5     for (int i = start; i < end; ++i)
6     {
7         int idx = i - start;
8         std::string s = guess + val[i];
9         std::copy(
10             s.begin(),
11             s.end(),
12             local_chars.begin() + local_offsets[idx]);
13     }
14 }
15 else{
16 #pragma omp parallel for
17     for (int i = start; i < end; ++i)
18     {
19         int idx = i - start;
20         std::copy(
21             val[i].begin(),
22             val[i].end(),
23             local_chars.begin() + local_offsets[idx]);
24     }
25 }

```

(4) 每个进程处理完了之后，我们现在开始收集结果，首先是先收集一些辅助信息：每个进程的猜测数量、每个猜测的长度、每个进程汇总字符串的总长度。

```

1 // 1. 收集每个进程的猜测数量
2 std::vector<int> counts(size);
3 MPI_Gather(&sub_num, 1, MPI_INT, counts.data(), 1, MPI_INT, 0,
4           MPI_COMM_WORLD);
5
6 // 2. 收集每个进程的每个猜测长度
7 std::vector<int> all_lengths; // 用于存储所有进程的猜测长度
8 std::vector<int> displs_lengths(size, 0); // 每个进程的猜测数量偏移量
9 if (rank == 0)
10 {
11     int total = 0;
12     for (int i = 0; i < size; ++i)
13     {
14         displs_lengths[i] = total;
15         total += counts[i];
16     }
17     all_lengths.resize(total);
18 MPI_Gatherv(sub_num > 0 ? local_lengths.data() : NULL, sub_num, MPI_INT,
19             all_lengths.data(), counts.data(), displs_lengths.data(),
20             MPI_INT, 0, MPI_COMM_WORLD);
21
22 // 3. 收集每个进程总字符的长度

```

```

23     int local_chars_size = local_chars.size();
24     std::vector<int> chars_counts(size);
25     MPI_Gather(&local_chars_size, 1, MPI_INT, chars_counts.data(), 1, MPI_INT
        , 0, MPI_COMM_WORLD);

```

(5) 收集每个进程的汇总字符串。

```

1     std::vector<int> displs_chars(size, 0);
2     std::vector<char> all_chars;
3     if (rank == 0)
4     {
5         int total = 0;
6         for (int i = 0; i < size; ++i)
7         {
8             displs_chars[i] = total;
9             total += chars_counts[i];
10        }
11        all_chars.resize(total);
12    }
13    MPI_Gatherv(local_chars_size > 0 ? local_chars.data() : NULL,
        local_chars_size, MPI_CHAR,
14        all_chars.data(), chars_counts.data(), displs_chars.data(),
15        MPI_CHAR, 0, MPI_COMM_WORLD);

```

(6) 在 rank0 进程里面把结果写入 guesses 中。这里我们还是先算偏移量，这样后面可以并行写入。

```

1     if (rank == 0)
2     {
3         size_t old_size = guesses.size();
4         size_t new_size = old_size + all_lengths.size();
5         guesses.resize(new_size);
6         // 先算所有偏移量
7         std::vector<size_t> all_offsets(all_lengths.size(), 0);
8         size_t acc = 0;
9         size_t a_l_size = all_lengths.size();
10        for (size_t i = 0; i < a_l_size; ++i)
11        {
12            all_offsets[i] = acc;
13            acc += all_lengths[i];
14        }
15        // 并行写入
16        #pragma omp parallel for schedule(static)
17        for (int i = 0; i < static_cast<int>(all_lengths.size()); ++i)
18        {
19            guesses[old_size + i] = std::string(
20                all_chars.data() + all_offsets[i],
21                all_lengths[i]);
22        }

```

23 }

4. 上面我们完成了对 TODO 部分的并行，但是我们在跑程序的时候发现即使我们写了 MPI 调用，程序跑完了依旧没有结束，在 debug 了一番之后，发现原来是只有 rank0 进程 break 出了 while 循环，而其他进程因为没有增加猜测数，所以一直在 while，导致程序一直运行无法结束，所以这里我们在 while 里面增加了一个阻塞，保证每次 while 的次数一致，并且当 rank0 退出 while 循环的时候，也能通知其他进程结束，退出循环。

每次到判断猜测数是否足够的时候，rank0 会给其他线程发送一个是否继续的信号，而其他线程也会在这个地方阻塞等待，直到收到信号。

```

1      if(rank == 0){
2          if (q.total_guesses - curr_num >= 100000)
3          {
4              cout << "Guesses_generated:" << history + q.total_guesses << endl;
5              curr_num = q.total_guesses;
6              int generate_n = 10000000;
7              if (history + q.total_guesses > 10000000)
8              {
9                  for (int dest = 1; dest < size; ++dest)
10                 {
11                     int terminate = -1;
12                     MPI_Send(&terminate, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
13                 }
14                 double end = MPI_Wtime();
15                 time_guess = end - start;
16                 cout << "Guess_time:" << time_guess - time_hash << "seconds" <<
17                     endl;
18                 cout << "Hash_time:" << time_hash << "seconds" << endl;
19                 cout << "Train_time:" << time_train << "seconds" << endl;
20                 break;
21             }
22             else{
23                 for (int dest = 1; dest < size; ++dest)
24                 {
25                     int terminate = 0;
26                     MPI_Send(&terminate, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
27                 }
28             }
29             else{
30                 for (int dest = 1; dest < size; ++dest)
31                 {
32                     int terminate = 0;
33                     MPI_Send(&terminate, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
34                 }
35             }
36         }
37         else{

```

```

38     int terminate;
39     MPI_Recv(&terminate, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
40             MPI_STATUS_IGNORE);
41     if(terminate == -1)
42     {
43         break;
44     }

```

5. main 函数结束前，结束 MPI 进程通信。

```

1     MPI_Finalize();

```

3. 运行结果

下面是运行结果，表2展示了不同版本在不同优化等级下的猜测时间及加速比。可以看到：

- 在未优化 (o0) 状态下，OPM 优化对原版性能提升较小 (约 1.04 倍)，而 MPI 优化反而导致性能下降，猜测时间变为原版的约 1.7 倍 (13.81 秒对 7.8 秒)。
- MPI 优化版本虽然期望并行加速，但其运行时间远高于原版，且随着优化等级的提升，性能反而进一步恶化，最高优化等级 o2 下，猜测时间甚至达到 7.46 秒，速度仅为原版的 0.08 倍，表明并行效果极差。

我们还记录了阻塞通信的时间，发现 O1 优化和 O0 的阻塞通信时间占比其实差不多，但是到了 O2 优化，发现阻塞时间大大增加。

表 1: 不同版本在不同优化下猜测时间对比及加速比

Version	原版 (s)	OPM 优化		MPI 优化	
		Time (s)	Speedup	Time (s)	Speedup
o0	7.80771	7.49629	1.04×	13.8104	0.57×
o1	0.612703	0.482152	1.27×	4.34495	0.14×
o2	0.630243	0.464643	1.36×	7.46154	0.08×

表 2: 不同优化下通信时间对比

Version	通信时间 (s)	通信开销 (%)
o0	2.15525	11.9557
o1	0.665428	12.0452
o2	3.76068	43.9852

综合两表分析：

1. MPI 优化版本因通信开销过大且负载或同步不均衡，导致整体性能大幅下降，在上面的实现代码中，一共用了五六次通信，而这是一次 popNext 里面的，所以实际上会有很多次通信，导致通信时间长，并行效率低。

2. 随着优化等级提升, 代码更激进的优化可能影响了通信代码的效率, 导致通信时间剧增, 通信阻塞成为性能瓶颈。
3. 该 MPI 实现存在通信与计算失衡问题, 需优化通信策略, 减少阻塞时间, 提升并行效率, 才能实现实际加速。

4. 阻塞时间差异探究

在前面的结果我们可以看到, O2 的阻塞通信时间比 O1 的多五六倍, 下面我们尝试研究一下为什么:

很有可能是 O2 相比于 O1 多做了一些优化, 导致 O2 反而效果差, 我们在编译时添加参数 `-fopt-info-vec-optimized -fopt-info-vec-missed -fopt-info-loop-optimized`, 来分别 O1/O2 的打印优化信息, 发现在 O2 优化中, 比 O1 多了一个优化: 把 `copy` 函数换成 `memcpy` 函数!!!

我们在编译的时候禁用这个优化, 发现 O2 的通信时间变成和 O1 差不多了, 结果如表 3

表 3: 不同优化下通信时间对比

版本	MPI	
	通信时间 (s)	通信开销 (%)
o0	2.15525	11.96
o1	0.665428	12.05
o2	3.76068	43.99
o2_v2	0.187089	3.79

注: o2_v2 指禁用 `memcpy` 优化后的版本

在大多数情况下, 使用 `memcpy` 替代显式循环拷贝操作通常能够带来更好的性能。这是因为现代编译器会将 `memcpy` 优化为底层高效的汇编指令 (如 SIMD 指令或 `rep movsb`), 从而减少指令数并提升缓存友好度。然而, 在本研究的 MPI 并行政程中, 我们观察到在开启 `-O2` 优化后, `std::copy` 被替换为 `memcpy`, 反而导致整体通信时间显著增加, 程序运行效率下降。对此现象, 我们分析其潜在原因如下。

首先, `memcpy` 优化带来了内存访问模式的显著变化。相较于 `std::copy` 的显式循环逐元素复制, `memcpy` 通常一次性使用高速缓存行进行大块数据传输, 可能采用非缓存写策略或特殊的预取指令。这种行为虽然减少了 CPU 指令数量, 但也可能导致写缓冲区和缓存层级的刷新时机集中, 从而对后续的 MPI 通信产生影响。

具体而言, 在 MPI 通信开始时, 进程需要保证本地内存中数据的可见性。如果数据写入采用了非缓存或延迟写策略, 内存屏障或缓存同步操作将被强制执行以保证数据一致性。这会导致某些进程在达到通信点时必须等待其他进程完成缓冲区刷新操作, 形成通信阻塞。

其次, `memcpy` 带来的高速数据传输使得进程间的同步差异被放大。执行 `memcpy` 的进程可能更早到达通信同步点, 而仍在使用显式循环的进程则较慢, 从而加剧了同步等待时间, 导致通信时间增长。

二、进阶要求

(一) 多 PT 并行实现

这个多 PT 我已经在多线程实现过一次了，框架改了很多，所以这里不再实现。

(二) guess、hash 流水线实现

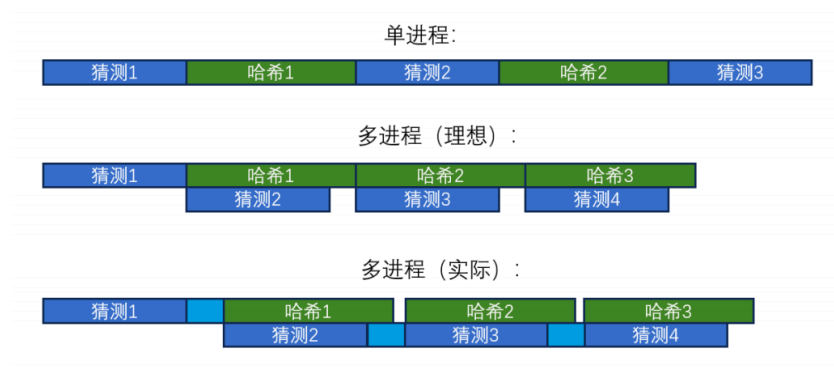


图 1: 流水线示例

1. 实现思路

我们这里流水线只用两个进程，分别用来做生成猜测和哈希结果，思路如下：

1. 先是两个进程同时进入 while 循环，然后先是 rank0 进程进行猜测生成。
2. 当猜测数量达到上限时，发送开始信号给 rank1 进程，然后这两个进程开始阻塞通信，rank0 进程发送辅助信息以及猜测字符串给 rank1 进程。
3. 当所有通信被接受之后，此时 rank0 进程继续生成猜测，rank1 进程开始哈希。
4. 如果 rank1 先完成了哈希，那么会阻塞在接收开始信号，等待 rank0 的猜测结果。如果 rank0 先完成，那么会等待 rank1 的接收下一次的开始信号。
5. 重复上面 2~4 过程，直到 rank0 不在生成猜测，此时 rank0 发送结束信号给 rank1，两个进程都 break 退出 while，结束 MPI 进程调用。

2. 代码实现

1. 首先进入循环的时候利用 if 语句进入不同的部分。

```

1  while (!q.priority.empty())
2  {
3      if(rank == 0){
4          ...    //guess部分
5      }
6      else if(rank ==1){
7          ...    //hash部分
8      }
9      else{//我们只用两个线程，其他线程直接退出while循环

```

```

10         break;
11     }
12 }

```

2. rank0 负责的 guess 部分:

(1) 先正常生成猜测, 直到到达猜测上限, 就发送给 rank1。先发送开始 hash 的信号; 然后发送总字符串以及一些辅助变量 (和基础部分一样, 这里我们选择加一个偏移量的辅助变量来实现 openmp 并行); 再清空现在存储的字符串, 更新历史数量。

```

1     if (curr_num > 1000000){
2         int start_signal = 1; // 发送哈希信号
3         MPI_Send(&start_signal, 1, MPI_INT, 1, 0, MPI_COMM_WORLD); // 发送
           开始信号
4
5         // 发送猜测字符串给另一个进程
6         // 发送字符串个数, 和总长度、发送每个字符串的长度
7         int num_guesses = q.guesses.size();
8         vector<int> lengths(num_guesses);
9         vector<int> offsets(num_guesses); // 用于存储每个字符串的偏移量
10        offsets[0] = 0; // 第一个字符串的偏移量为0
11        int total_length = 0;
12    #pragma omp parallel for num_threads(7) shared(q, lengths) // 使用 OpenMP 并
           行化
13        for (int i = 0; i < num_guesses; i++) {
14            lengths[i] = q.guesses[i].length();
15        }
16        for (int i = 0; i < num_guesses; i++) {
17            offsets[i] = total_length; // 记录每个字符串的偏移量
18            total_length += lengths[i];
19        }
20        //发送总字符串
21        vector<char> all_guesses(total_length + 1);
22        all_guesses[total_length] = '\0';
23    #pragma omp parallel for num_threads(7) shared(all_guesses, q, offsets)
24        for (int i = 0; i < num_guesses; i++)
25        {
26            memcpy(&all_guesses[offsets[i]], q.guesses[i].data(), lengths[i])
                ;
27        }
28        // 发给进程1, 一起发
29        MPI_Send(&num_guesses, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
           // 发送字符串个数
30        MPI_Send(&total_length, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
           // 发送总长度
31        MPI_Send(lengths.data(), num_guesses, MPI_INT, 1, 0, MPI_COMM_WORLD
           ); // 发送每个字符串的长度
32        MPI_Send(all_guesses.data(), total_length, MPI_CHAR, 1, 0,
           MPI_COMM_WORLD); // 发送所有字符串

```

```

33
34     //恢复数据，记录已经生成的口令总数
35     history += curr_num;
36     curr_num = 0;
37     q.guesses.clear();
38 }

```

(2) 当猜测数量足够了，我们就发送结束信号给 rank0，同时记录时间，因为这里 hash 和 guess 是流水线的并行，所以时间是重叠的，这里面我们记录的时间是这个流水线的总时间。

```

1  if (q.total_guesses - curr_num >= 100000)
2  {
3      cout << "Guesses_generated:" << history + q.total_guesses << endl;
4      curr_num = q.total_guesses;
5
6      // 在此处更改实验生成的猜测上限
7      int generate_n = 10000000;
8      if (history + q.total_guesses > 10000000)
9      {
10         //发信号告诉rank1，结束程序
11         int end_signal = -1;
12         MPI_Send(&end_signal, 1, MPI_INT, 1, 0, MPI_COMM_WORLD); // 发送
            结束信号
13         //当哈希结束的时候，它才会收到，这里是阻塞的，所以肯定一致
14         double end = MPI_Wtime();
15         time_guess = end - start;
16         cout << "Guess+Hash time:" << time_guess << "seconds" << endl;
17         // cout << "Hash time:" << time_hash << "seconds" << endl;
18         cout << "Train time:" << time_train << "seconds" << endl;
19         break;
20     }
21 }

```

3. rank1 负责的 hash 部分:

(1) 先阻塞等待 rank0 发送信息，一开始都是在等 rank0 发送 1 信息，也就是开始哈希；到最后的时候收到的值是-1，就是结束哈希的信号。这里需要转一下脑子的是，在源代码里面，最后一批生成的猜测是不需要做哈希的！其实这里浪费了一批猜测，要不浪费了时间，要不就是浪费了结果。所以这里接受信号的位置是同一个。

```

1  // 接收开始或结束信息
2  int signal;
3  MPI_Recv(&signal, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
4  if (signal == -1)
5      break;

```

(2) 剩下部分就比较简单了，就是接收通信，然后哈希，这里仍然用计算偏移量来用 openmp 并行加速

```

1 // 接收开始或结束信息
2 int signal;
3 MPI_Recv(&signal, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
4 if (signal == -1)
5     break;

```

3. 结果分析

结果如表4所示, 我们对不同优化级别下的 Guess+Hash 整体时间进行了对比和加速比分析。可以看到, 直接使用 -O1 或 -O2 优化在初始实验中反而出现了负加速现象

表 4: Guess+Hash 时间对比与加速比分析

优化级别	原版			Pipeline		加速比
	Guess(s)	Hash(s)	Guess+Hash(s)	Guess+Hash(s)	Train(s)	
o0	7.80771	9.61899	17.4267	14.1419	96.636	0.81×
o1	0.612703	3.11771	3.730413	7.09897	28.7828	0.53×
o2	0.630243	3.13003	3.760273	7.47602	25.909	0.50×
o1_v2	-	-	-	6.89462	30.1454	0.54×
o2_v2	-	-	-	3.03676	28.6844	1.24×

注: 加速比 = 原版 (Guess+Hash) 时间 / Pipeline (Guess+Hash) 时间;

为进一步分析性能瓶颈, 我们采用了逐步排查和对比的策略:

(1) strcpy 优化对性能的影响 通过对比编译器的优化信息 (-fopt-info-vec-missed -fopt-info-loop-optimized), 发现 O2 默认对 strcpy() 调用进行了循环分发和完全展开优化。这一优化在单核情况下可能有利, 但在 MPI 分布式环境中反而显著增加了内存带宽和 cache 压力, 导致通信和数据拷贝性能下降。

(2) 禁用 strcpy 展开后的改进 我们在 O2 版本中通过禁用相关优化, 成功避免了该负优化。经过调整后的 O2_v2 版本, Guess+Hash 时间从 7.48 秒大幅下降到仅 3.04 秒, 实现了 1.24× 的加速比, 这说明优化策略的选择对性能影响极大。

相比之下, O1 版本本身对 strcpy 并未进行激进的循环优化, 因此在 O1_v2 版本禁用相关优化后, 性能没有显著变化。

(3) 小结 令人欣喜的是, 在仅使用两个进程的条件下, 通过合理调整编译优化策略, 我们仍然在 O2_v2 版本上成功实现了性能提升。相比原版串行程序, 该版本在 Guess+Hash 阶段实现了 24% 的加速, 证明了 Pipeline 思路在适当条件下是可行的。

三、 总结

本文针对口令猜测任务, 基于 ARM 架构实现了 MPI 并行优化方案, 并结合 OpenMP 和 SIMD 技术进行了多层次性能提升研究, 主要结论如下:

1. **基础 MPI 实现**: 通过任务划分和 MPI 通信, 实现了猜测生成的并行化, 但因频繁的阻塞通信 (5-6 次/循环) 导致性能下降。O2 优化下, 通信时间占比高达 43.99%, 加速比仅 0.08 $\times$ 。禁用 `memcpy` 优化后, 通信时间从 3.76 秒降至 0.19 秒, 显著缓解阻塞瓶颈。
2. **流水线优化**: 采用两进程流水线设计, 分别负责猜测生成和哈希计算, 结合 OpenMP 并行化字符串处理。在 O2 优化 (禁用 `memcpy`) 下, Guess+Hash 时间从 3.76 秒降至 3.04 秒, 加速比达 1.24 $\times$, 验证了流水线并行的有效性。
3. **性能瓶颈分析**: 通信与计算失衡是 MPI 实现的主要瓶颈。优化编译 (如 O2) 可能引入不利于通信的内存访问模式 (如 `memcpy`), 需谨慎选择优化策略。
4. **改进方向**: 未来可通过非阻塞通信 (如 `MPI_Isend`)、动态任务划分和双缓冲机制进一步减少通信开销, 提升并行效率。

综上, 本实验验证了 MPI 流水线结合 OpenMP 和 SIMD 的多层次并行优化在口令猜测任务中的应用价值, 为高性能并行计算提供了实践参考。